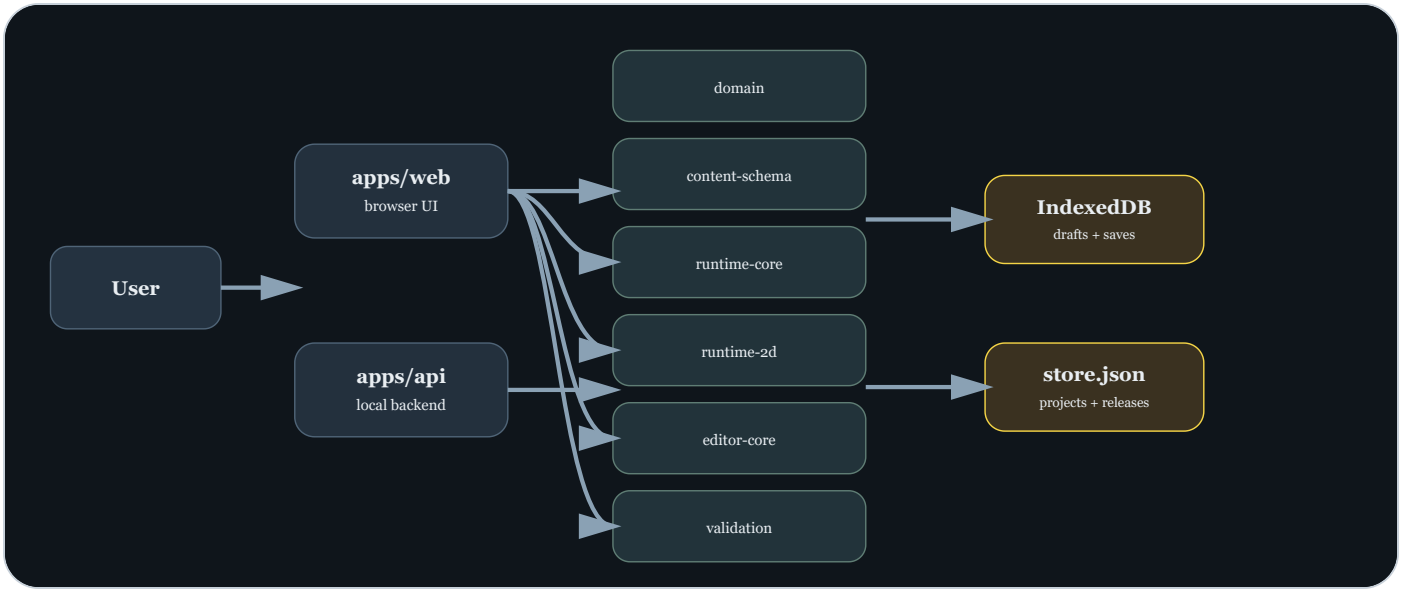
ACS MILESTONE 9

System Reference

A technical map of how ACS fits together, with concrete end-to-end flows for runtime movement, inspection, editor tile painting, entity placement, validation, persistence, and rendering.

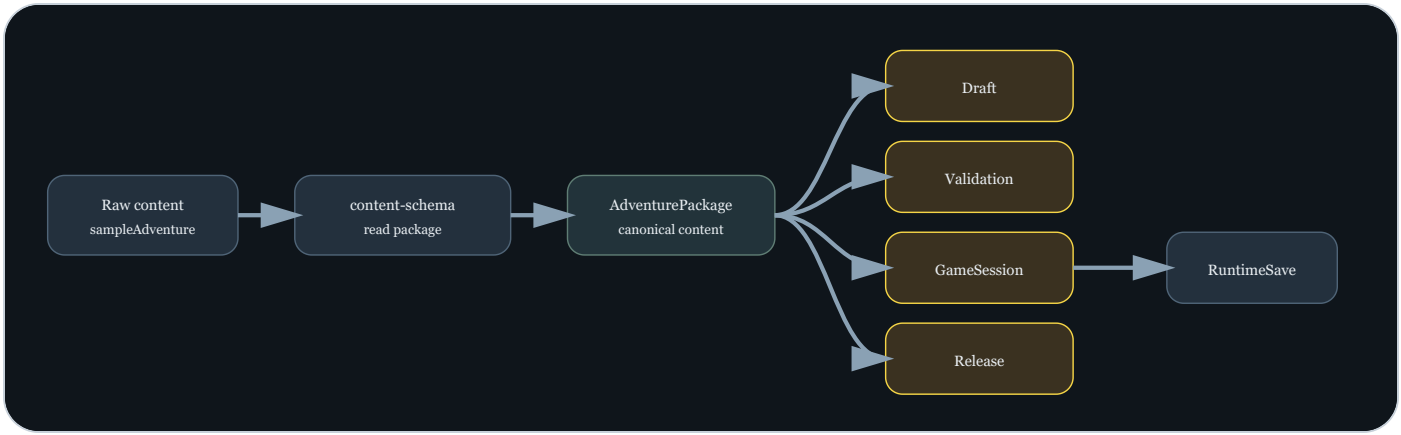
Companion documents: the user guide explains how to use the application; this system reference explains what happens internally. The Markdown source includes Mermaid diagrams, and this PDF includes readable rendered equivalents plus screenshots.

High-Level Architecture



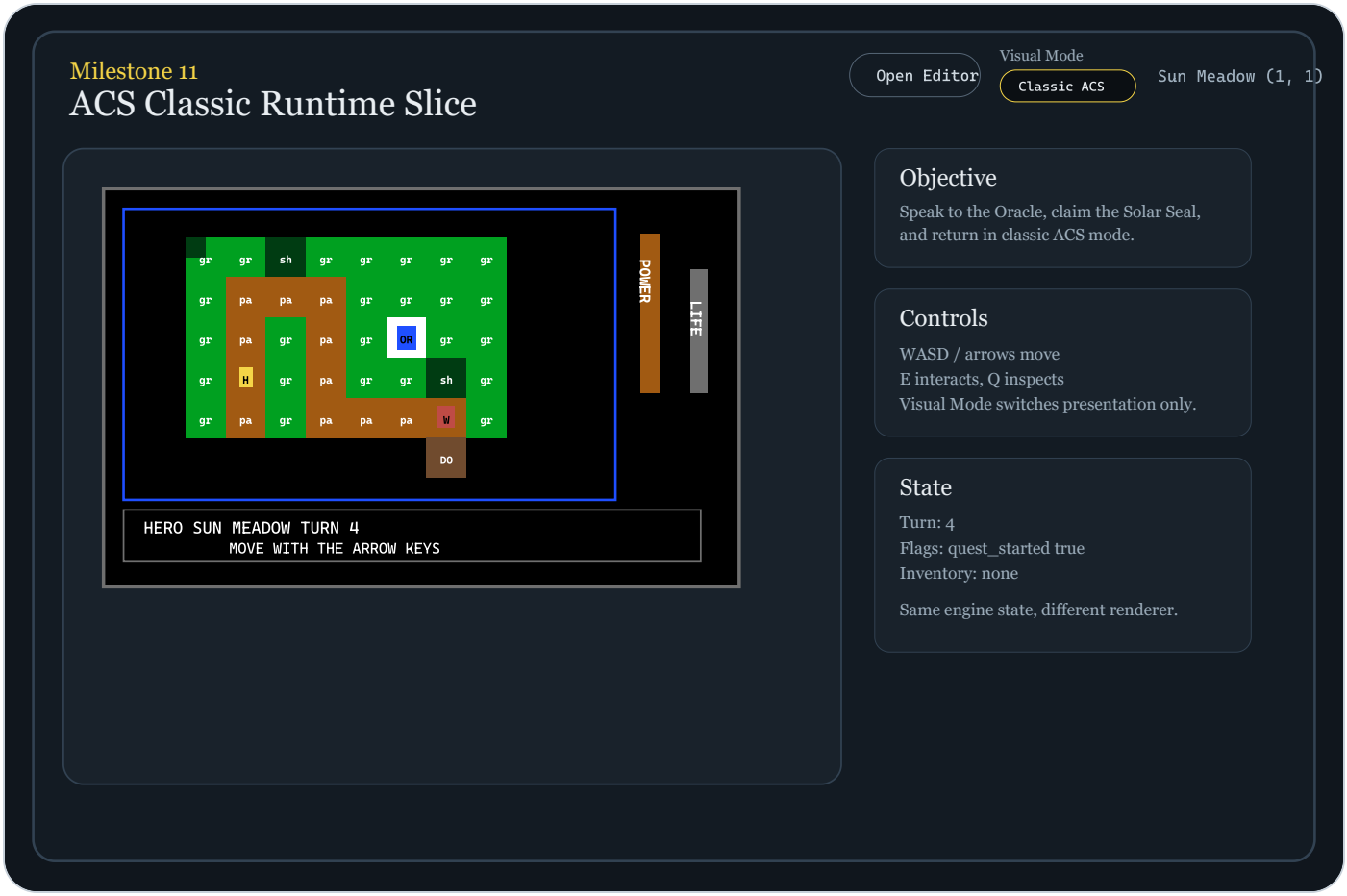
Area	Package or app	Responsibility
Shared vocabulary	`packages/domain`	IDs, maps, entities, triggers, dialogue, actions, and conditions.
Content ingestion	`packages/content-schema`	Reads raw authored content and normalizes it into an `AdventurePackage`.
Simulation	`packages/runtime-core`	Commands, state mutation, triggers, dialogue, enemy turns, snapshots, and events.
Rendering	`packages/runtime-2d`	Draws runtime state to canvas without owning game rules.
Editing	`packages/editor-core`	Pure draft operations such as `setTileAt`, `addEntityInstance`, and `moveEntityInstance`.
Validation	`packages/validation`	Publish-readiness checks shared by browser and API.
Persistence	`packages/persistence`	IndexedDB runtime saves and local editor drafts.
Backend	`apps/api`	Projects, releases, validation endpoint, and file-backed storage.

Data Model Layers



Key distinction: a release is immutable content, while a save is runtime state. A save wraps the engine's existing `RuntimeSnapshot`, so persistence does not invent a second save-game model.

Current Screens



MILESTONE 9

ACS Construction Set MVP

Back To Play | Save Draft | Reset Draft | Playtest

Map: Inner Shrine
Mode: Entities
Move Entity: entity_wolf
Add Definition: Shrine Wolf
Place New

Active Brush
Place Shrine Wolf

stone	stone	stone	stone	stone	stone
stone	floor	floor	floor	floor	stone
stone	floor	floor	floor	altar	stone
stone	floor	floor	floor	floor	stone
stone	floor	floor	floor	floor	stone
stone	door	stone	stone	stone	stone

Placing Shrine Wolf on Inner Shrine. Placement: multiple. Click a cell to add an instance.

Metadata

Title
Oracle of the Solar Seal

Description
A tiny milestone slice with movement, dialogue, triggers, enemy behavior, and map travel.

Project & Release

Connected to local API as Local Designer.
No project linked yet.
No server validation run yet.

Validate Draft | Create Project | Save Project | Publish Release

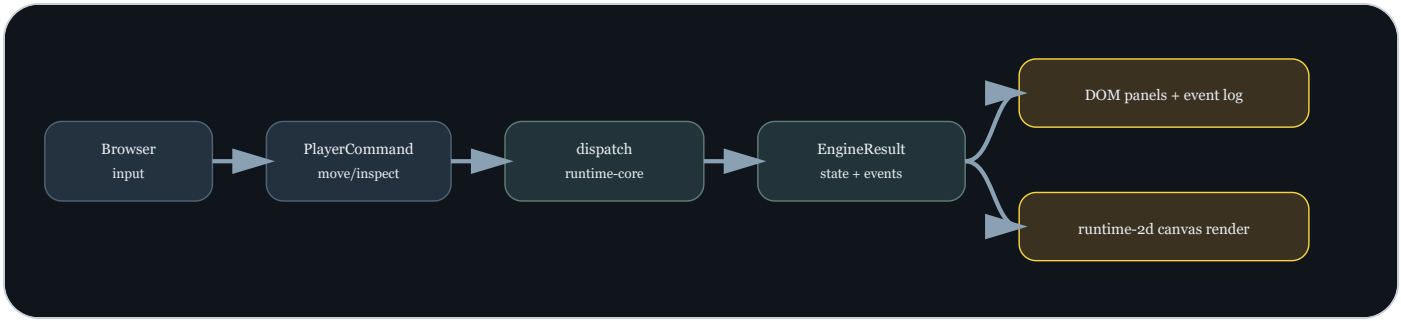
Validation

Draft is publishable with 0 warnings.
No validation issues.

Entities On Map

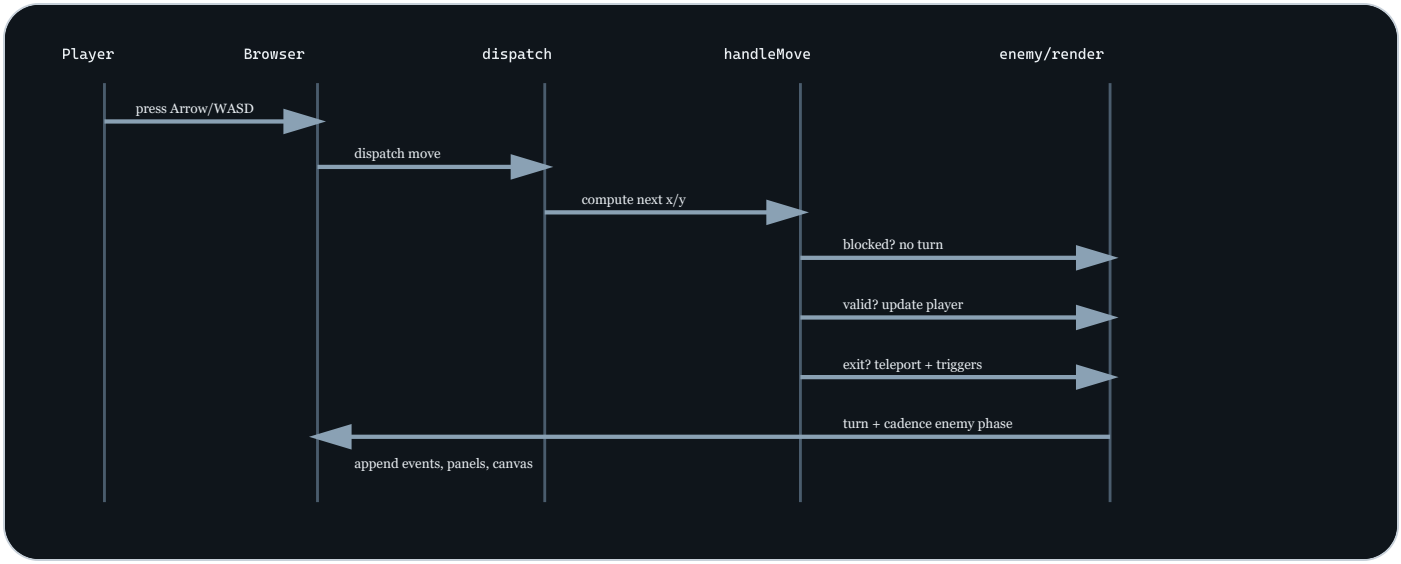
entity_wolf: Shrine Wolf at (2, 2)
entity_wolf_1: Shrine Wolf at (3, 2)

Runtime Input To Rendering



Runtime input follows a narrow path: `apps/web/src/index.ts` turns browser input into a `PlayerCommand`, `runtime-core` mutates session state and emits events, then the browser updates panels and asks `runtime-2d` to redraw.

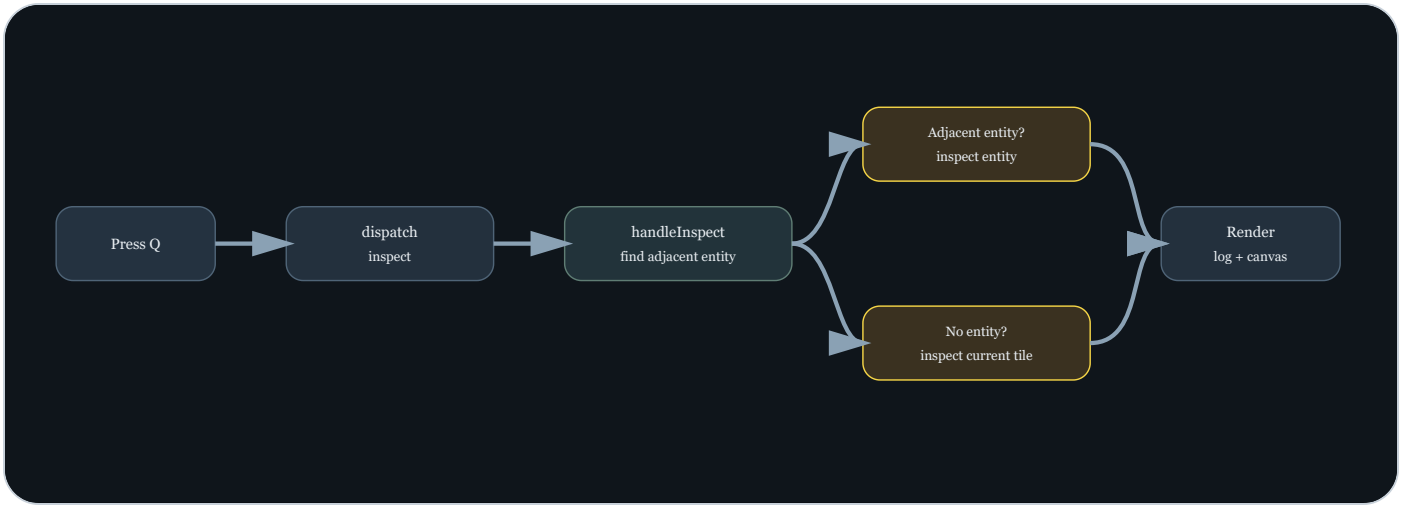
Use Case 1: Move Command



1. The keydown handler in `apps/web/src/index.ts` maps Arrow keys and `WASD` to `{ type: "move", direction }`.
2. `runCommand` calls `session.dispatch(...)`, appends returned events, and calls `renderEverything`.
3. `dispatch` calls `handleMove` and uses the returned boolean to decide whether the command consumed a turn.
4. `handleMove` computes the next coordinate, rejects bounds or occupied destinations, or updates `state.player`.
5. If the destination is an exit, the engine changes `currentMapId`, teleports the player, and runs map-load and tile triggers.
6. If movement succeeded, `dispatch` increments `state.turn`, emits `turnEnded`, and runs the enemy phase.
7. The browser updates map name, player position, flags, inventory, dialogue, event log, and canvas.

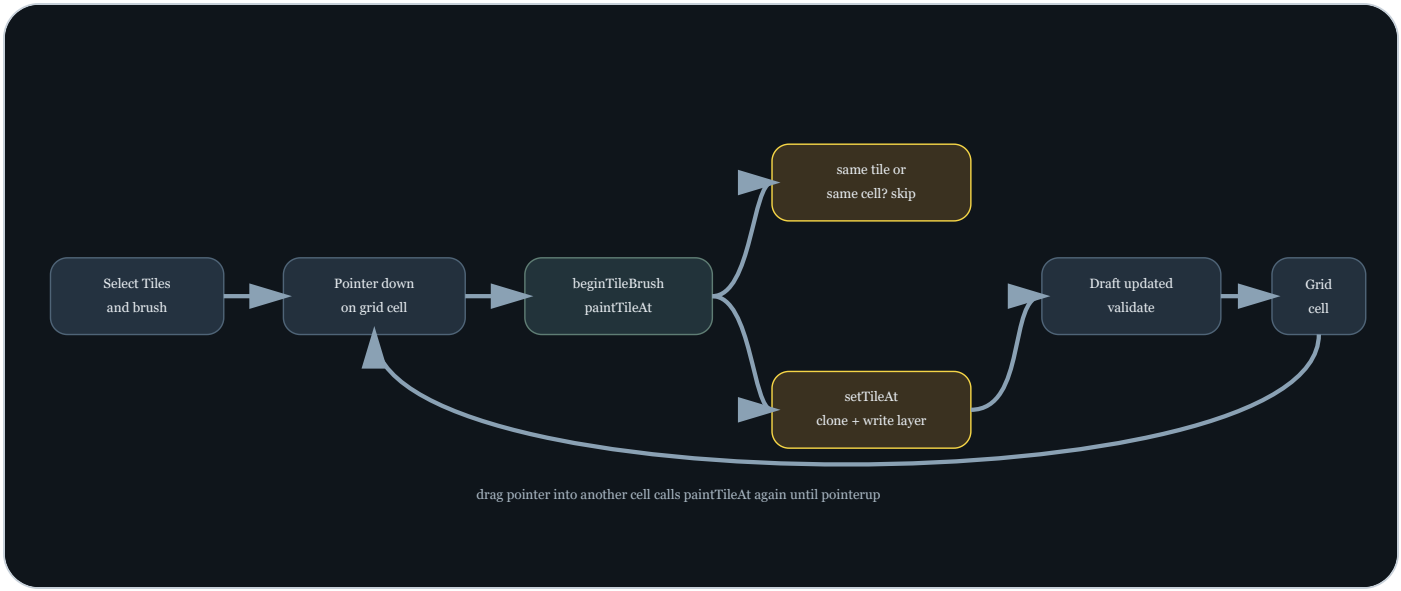
Current behavior: blocked movement does not consume a turn and does not grant enemies a free action. Enemy behavior profiles may define `turnInterval`; the sample Shrine Wolf uses `turnInterval: 3`, so it only acts on every third successful player turn.

Use Case 2: Inspect Command



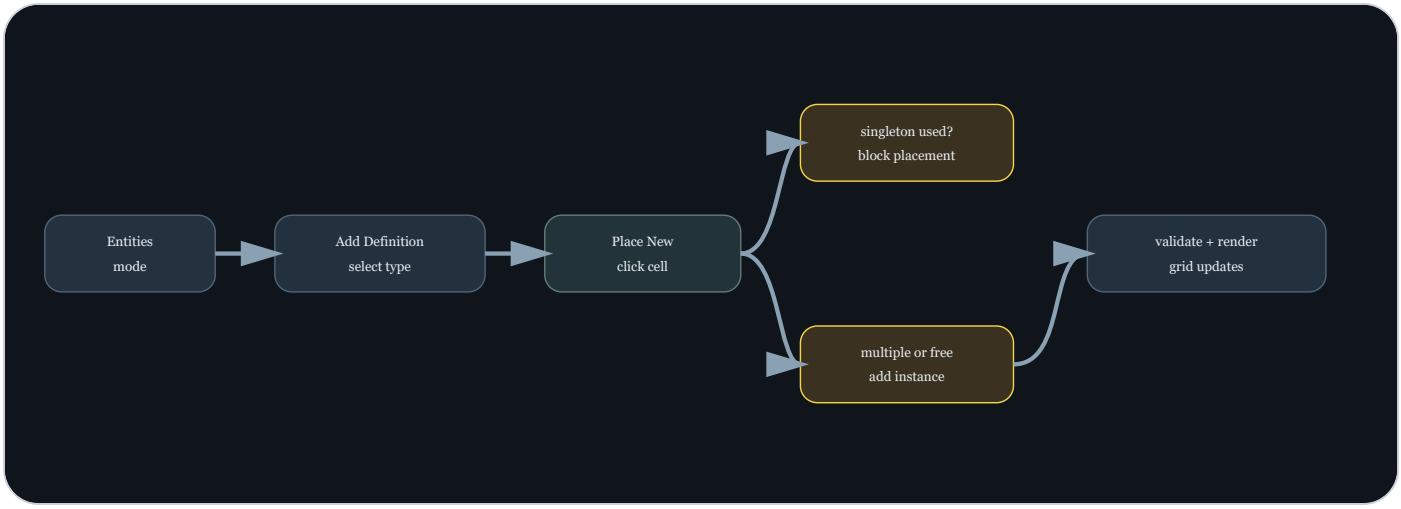
1. The browser maps `Q` to `{ type: "inspect" }`.
2. `dispatch` calls `handleInspect(undefined, events)` and treats a completed inspection as a turn-consuming action.
3. With no explicit direction, `findEntityInDirection` searches for the first active adjacent entity on the current map.
4. If an adjacent entity exists, the engine emits `inspectResult` for that entity.
5. If no adjacent entity exists, the engine emits an `inspectResult` describing the player's current coordinate and map id.
6. Inspect does not directly move the player, grant items, set flags, or start dialogue.
7. `dispatch` increments `state.turn`, emits `turnEnded`, then runs the enemy phase. Enemies still respect their configured `turnInterval`.

Use Case 3: Editor Tile Brush



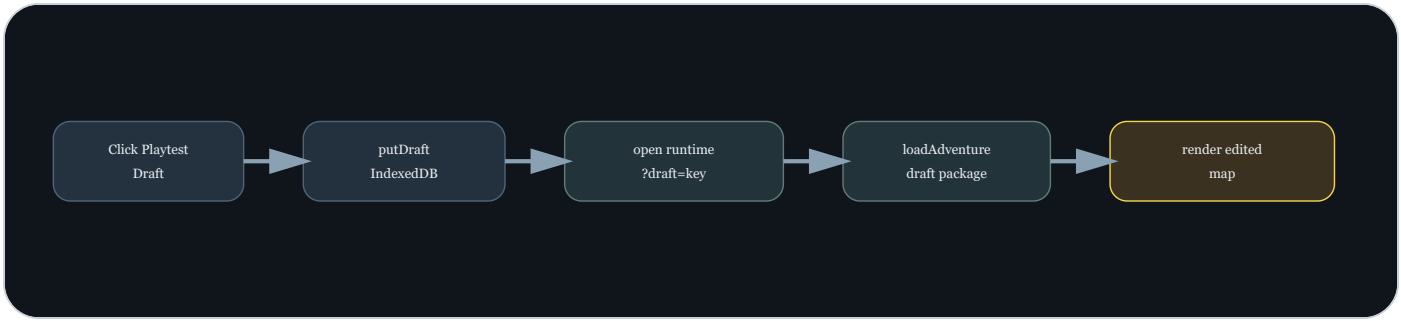
1. The designer selects `Tiles` mode and picks a tile id. The browser stores it in `selectedTileId`.
2. `renderBrushPreview` updates the visible active brush.
3. Each grid cell has a `pointerdown` handler in tile mode. Pressing a cell calls `beginTileBrush(x, y)`.
4. `beginTileBrush` sets `isTileBrushActive = true`, clears the duplicate-cell guard, and calls `paintTileAt`.
5. `paintTileAt` skips if the editor is not in tile mode, if the same cell was already painted in this drag, or if the cell already has the selected tile.
6. If the tile differs, the browser calls `setTileAt(draft, currentMapId, x, y, tileId)` from `editor-core`.
7. `setTileAt` clones the package, finds the map and active layer, calculates `index = y * map.width + x`, writes the tile id, and returns a new draft.
8. The browser refreshes the changed grid cell, reruns local validation, and updates project/publish controls.
9. Dragging into another cell calls `paintTileAt` through `pointerenter`. Pointer up ends the brush.

Use Case 4: Editor Entity Placement



1. The designer switches to `Entities` mode.
2. The editor shows `Move Entity`, `Add Definition`, and `Place New`.
3. Choosing a reusable definition stores `selectedEntityDefinitionId` and enters new-instance placement mode.
4. `singleton` definitions are blocked after one instance exists anywhere in the adventure.
5. `multiple` definitions can be placed repeatedly.
6. `addEntityInstance` clones the package, creates a unique id, pushes an `EntityInstance`, then validation and grid rendering rerun.

Editor To Playtest



Painting a tile changes only the editor draft. To see it in play, the editor saves the draft to IndexedDB and opens the runtime with a draft query parameter. The runtime loads that same `AdventurePackage`, so there is no separate editor-only map format.

Validation, Publishing, Save, And Load

Validation and publishing

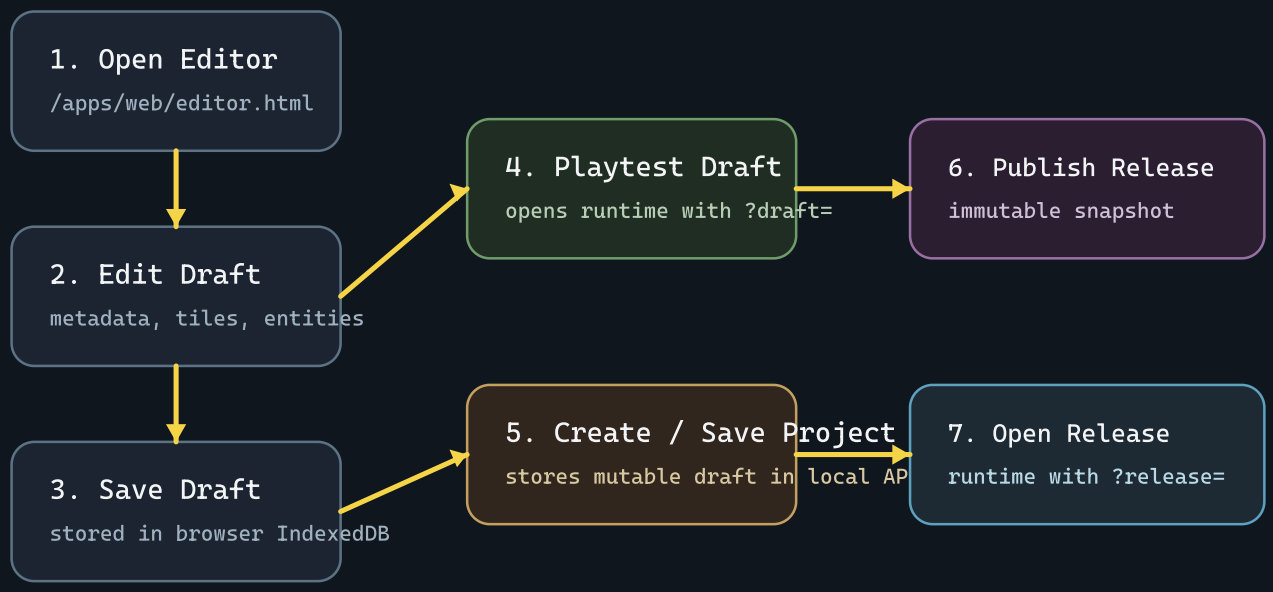
1. Local edits rerun `validateAdventure(draft)`.
2. `Validate Draft` calls the API validation endpoint.
3. Project save and publish are disabled when local validation is blocking.
4. The API validates again before creating an immutable release.

Runtime save/load

1. `Save` calls `session.serializeSnapshot()`.
2. `persistence.saveSession` stores the snapshot in IndexedDB.
3. `Load` retrieves the record and calls `engine.loadAdventure(activeAdventure, snapshot)`.
4. The browser rerenders from restored state.

ACS User Workflow

How the current application is used from editing to playtesting to publishing.

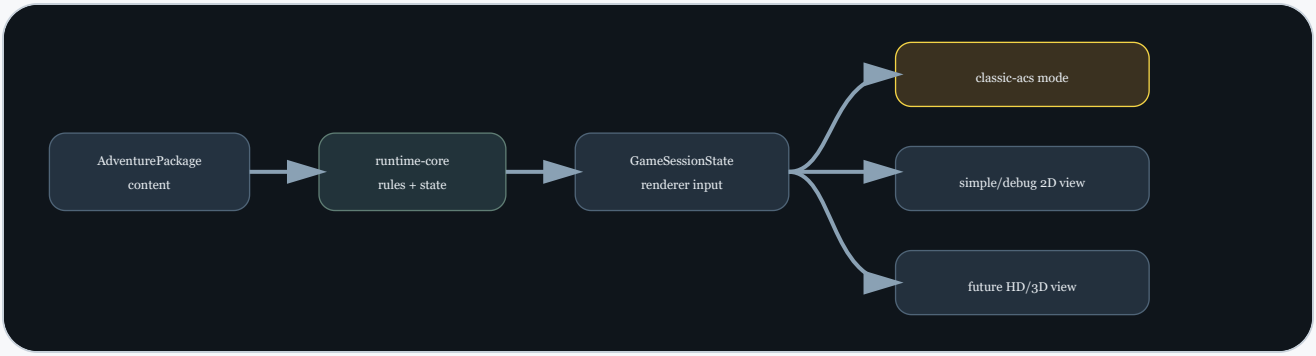


Forward Milestone Path: Classic ACS Feel

The project should deliberately capture the feel of the original 1980s Adventure Construction Set while keeping the engine free of renderer assumptions. The legacy image set points to two major tracks: a classic gameplay panel and deeper construction-set authoring tools.

Classic runtime presentation

The classic runtime mode should be a presentation layer over the same `AdventurePackage`, `RuntimeSnapshot`, and `GameSessionState` used by the current renderer.



- fixed-aspect vintage gameplay panel
- tile/icon map viewport on a dark field
- right-side status rail for life, power, and future actor resources
- bottom message band for location names, prompts, interaction text, and command hints
- pixelated sprite scaling and controlled palette
- asset IDs resolved through manifests so the same map can later render with HD or 3D assets

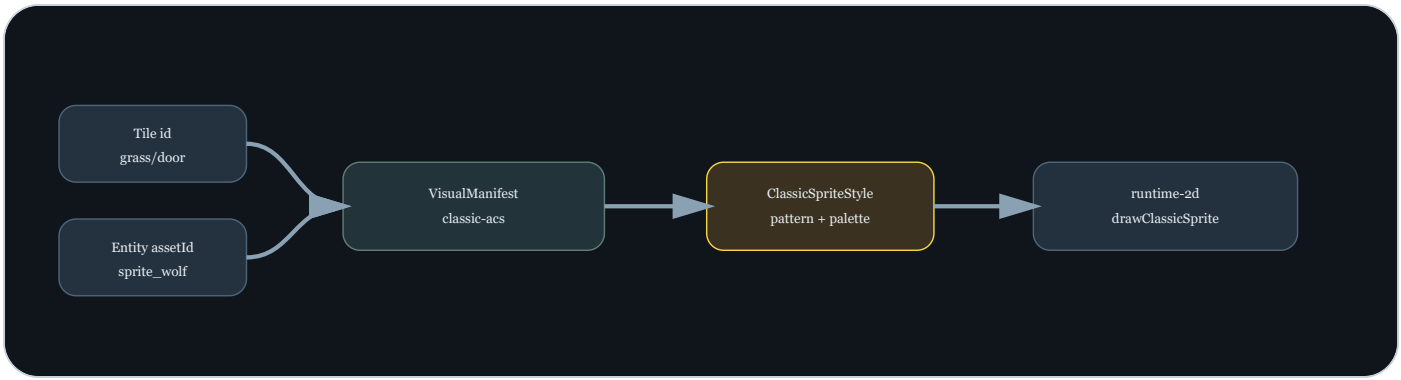
Classic editor capability track

The original editor suggests future modes for terrain pictures, thing pictures, creature pictures, map/floor wiring, reusable definitions, actor profiles, possessions, triggers, portals, text, and dialogue.

1. **Milestone 10:** completed `classic-acs` visual mode with a gameplay viewport, right status rail, bottom message band, and renderer theme switching.
2. **Milestone 11:** completed sprite manifests, a first classic asset set, entity sprite IDs, manifest-driven classic renderer lookup, and validation warnings for missing sprite references.
3. **Milestone 12:** next, add definition editors for entities, items, tiles, terrain, sprites, placement rules, stats, and behavior metadata.
4. **Milestone 13:** add thing, trigger, portal, text, and dialogue editors using structured rules/actions.
5. **Milestone 14:** add map scale and adventure-structure tools for world, region, local, interior, and dungeon-floor maps.
6. **Milestone 15:** add richer character/profile and possession systems, with runtime status rendering and editor support.

Milestone 11 Sprite Manifests

Milestone 11 moves the classic visual mode from hardcoded tile and entity assumptions toward manifest-driven presentation data. `runtime-core` remains renderer-agnostic; `runtime-2d` reads the adventure's `classic-acs` visual manifest when deciding how to draw canvas sprites.



- `AdventurePackage.visualManifests` stores renderer-facing presentation metadata outside the simulation rules.
- The sample adventure defines `Classic Solar Seal Sprite Set` with tile entries for grass, path, shrub, stone, floor, altar, door, water, and void.
- The Hero, Oracle, and Shrine Wolf definitions now point at stable sprite IDs through `assetId`.
- The classic renderer resolves a tile/entity sprite from the manifest first, then falls back to built-in defaults for older packages.
- Validation warns when classic sprite references are missing without blocking the project, because fallbacks keep content playable.

Extension Points

- A higher-resolution renderer can live beside `runtime-2d` as long as it consumes `AdventurePackage` and `GameSessionState`.
- A future 3D renderer should not change browser input or editor logic unless the content model needs richer spatial metadata.
- Real-time play would most likely evolve the command scheduling, enemy `turnInterval`, and enemy phase in `runtime-core`.
- Richer enemy AI belongs in `runtime-core` or a future AI package, not in the DOM or canvas renderer.
- New editor creation tools should start as pure functions in `editor-core`, then be wired into `apps/web/src/editor.ts`.